CSci 21 Data Structures and Algorithms
Module 5

Queues

Learning Outcomes

LO 5.1 Implement accurate queue operations

LO 5.2 Implement accurate solutions for problems using queue ADT

LO 5.3 Diagnose queue-specific errors

Setting the Expectations

PREREQUISITES

- Utilized **arrays** and **linked lists** in programming solutions.
- Utilized **pointers** in **C/C++.**

What's a Queue?

First-in First-Out (FIFO)
linear data structure

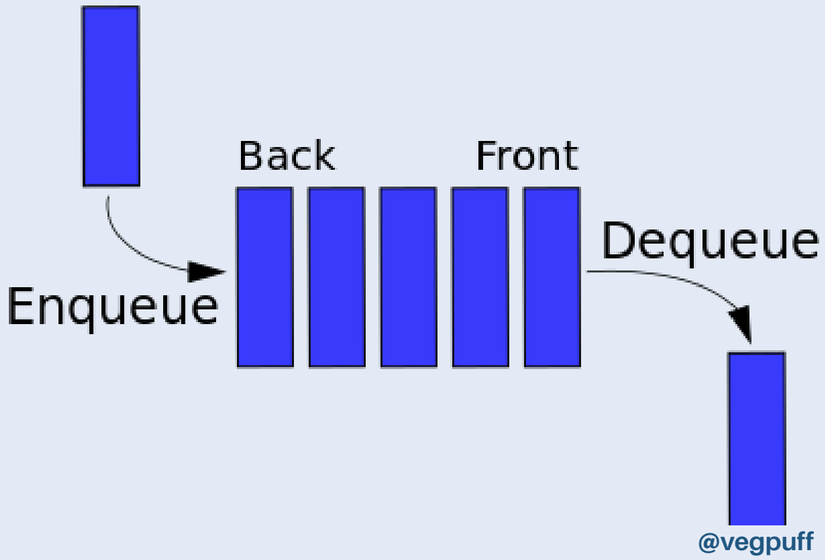
Types of Queue

Bounded queue

Priority queue

Unbounded queue

Main Operations



Auxiliary Operations

getFront returns the element at the front of the queue

IsEmpty returns true if the queue is empty and false otherwise

IsFull returns true if the queue is full and false otherwise

Implement
Queue and Priority Queue

Solve Problems Involving **Queue**

Complexity of Queue Operations

OPERATION	TIME COMPLEXITY	WHY?
enqueue	O(1)	array or circular linked list insert at the end/tail
	$O(\log_2 n)$	priority queue as a binary heap
dequeue	O(1)	remove the head node
	$O(\log_2 n)$	priority queue as a binary heap
getFront	O(1)	direct access to the front of the array/head of the list
isEmpty	O(1)	ask if size = 0 or head is null
isFull	O(1)	ask if size = capacity

Queue-specific Errors

1. **Queue Underflow**
   - ❌ Calling dequeue() or getFront() on an empty queue → Crash!
   - ✅ Always check isEmpty() first.

2. **Queue Overflow (Bounded queue)**
   - ❌ Pushing onto a full queue
   - ✅ Check isFull() before enqueue.

3. **Circular Buffer Index Errors**
   - ❌ Incorrect modulo arithmetic → data corruption
   - ✅ Use: **rear = (rear + 1) % capacity** and **front = (front + 1) % capacity**.

"The key to wisdom is constant and frequent questioning, for by doubting we are led to question, by questioning we arrive at the truth..."

- Peter Abelard

OPEN FORUM Q/A

for Module 5